Document Number:	**P0670R4**, ISO/IEC JTC1 SC22 WG21
Audience:	CWG, LWG
Date:	2018-06-08
Authors:	Matúš Chochlík (chochlik@gmail.com)
	Axel Naumann (axel@cern.ch)
	David Sankel (camior@gmail.com)

Function reflection

0. Introduction

P0194 introduced static reflection for types and variables. This paper adds static reflection of functions.

Reflection proposed here behaves as follows:

The functionality introduced here allows for reflection of calls of concrete functions. This enables, for instance, GUI generation, building a catalogue for remote procedure calls, documentation generation, and signal/slot frameworks. Like P0194, this proposal omits attributes, templates, and reification (i.e. conversion of a meta object to the base level): all warrant a separate paper. We would welcome a paper especially on static reflection of attributes, matching the interface-style of P0194 and this paper! Linkage and friends will be part of a follow-up paper to P0194; they will have a combined "effect" on P0194 and this paper.

0.1 Interplay with other proposals

Most notably, this proposal relies on the Reflection TS and the Concepts TS.

P0385 discusses use cases, rationale, design decisions, and the future evolution of the proposed reflection facility. It also has usage examples and replies to frequently asked questions.

1 Scope [intro.scope]

This document is written as a set of changes against the Reflection TS (N4746). Instructions to modify or add paragraphs are written as explicit instructions. Modifications made directly to existing text from the Reflection TS use underlining to represent added text and ~~strikethrough~~ to represent deleted text.

2 Normative references [intro.refs]

No changes are made to Clause 2 of the Reflection TS.

3 Terms and definitions [intro.defs]

No changes are made to Clause 3 of the Reflection TS.

4 General [intro]

4.3 Acknowledgements [intro.ack]

Modify the section as follows:

> This work is the result of a collaboration of researchers in industry and academia. We wish to thank people who made valuable contributions within and outside these groups, including Ricardo Fabiano de Andrade, Roland Bock, Chandler Carruth, Jackie Kay, Klaim-Joël Lamotte, Jens Maurer, and many others not named here who contributed to the discussion.

5 Lexical conventions [lex]

No changes are made to Clause 5 of the Reflection TS.

6 Basic concepts [basic]

6.2 One-definition rule [basic.def.odr]

In C++ [basic.def.odr], insert a new paragraph after the existing paragraph 8:

> A function or static variable reflected by [dcl.type.reflexpr] is odr-used by the specialization (21.11.4.10, 21.11.4.18), as if by taking the address of an *id-expression* nominating the function or variable.

In C++ [basic.def.odr], insert a new bullet (12.2.3) after (12.2.2):

> or
>
> - a type implementing (21.11.3.1), as long as all operations (21.11.4) on this type yield the same constant expression results.

7 Standard conversions [conv]

No changes are made to Clause 7 of the Reflection TS.

8 Expressions [expr]

8.4 Primary expressions [expr.prim]

8.4.5 Lambda expressions [expr.prim.lambda]

8.4.5.2 Captures [expr.prim.lambda.capture]

In C++ [expr.prim.lambda.capture], apply the following change to paragraph 7:

> If an expression potentially references a local entity within a declarative region in which it is odr-usable, and the expression would be potentially evaluated if the effect of any enclosing expressions (8.5.1.8) or use of a *reflexpr-specifier* (10.1.7.6) were ignored, the entity is said to be *implicitly captured* by each intervening *lambda-expression* with an associated *capture-default* that does not explicitly capture it.

In C++ [expr.prim.lambda.capture], apply the following change to paragraph 11:

> Every *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use (6.2) of an entity captured by copy, as well as every use of an entity captured by copy in a *reflexpr-operand*, is transformed into an access to the corresponding unnamed data member of the closure type.

8.5 Compound expressions [expr.compound]

8.5.1 Postfix expressions [expr.post]

In C++ [expr.post], apply the following change:

> *postfix-expression* :
> *primary-expression*
> *postfix-expression* [*expr-or-braced-init-list*]
> ~~*postfix-expression* (*expression-list*$_{opt}$)~~
> *function-call-expression*
> ~~*simple-type-specifier* (*expression-list*$_{opt}$)~~
> ~~*typename-specifier* (*expression-list*$_{opt}$)~~
> ~~*simple-type-specifier braced-init-list*~~
> ~~*typename-specifier braced-init-list*~~
> *functional-type-conv-expression*
> *postfix-expression* . $_{opt}$ *id-expression*
> *postfix-expression* -> $_{opt}$ *id-expression*
> *postfix-expression* . *pseudo-destructor-name*
> *postfix-expression* -> *pseudo-destructor-name*
> *postfix-expression* ++
> *postfix-expression* --
> < *type-id* > (*expression*)
> < *type-id* > (*expression*)
> < *type-id* > (*expression*)
> < *type-id* > (*expression*)
> (*expression*)

(*type-id*)

function-call-expression :
postfix-expression (*expression-list*$_{opt}$)

functional-type-conv-expression :
simple-type-specifier (*expression-list*$_{opt}$)
typename-specifier (*expression-list*$_{opt}$)
simple-type-specifier braced-init-list
typename-specifier braced-init-list

expression-list :
initializer-list

9 Statements [stmt.stmt]

No changes are made to Clause 9 of the Reflection TS.

10 Declarations [dcl.dcl]

10.1 Specifiers [dcl.spec]

10.1.7 Type specifiers [dcl.type]

10.1.7.2 Simple type specifiers [dcl.type.simple]

In C++ [dcl.type.simple], apply the following change

...
reflexpr-operand:

type-id
nested-name-specifier$_{opt}$ *identifier*
nested-name-specifier$_{opt}$ *simple-template-id*
(*expression*)
function-call-expression
functional-type-conv-expression

10.1.7.6 Reflection type specifier [dcl.type.reflexpr]

Apply the following modification to the enumeration:

- ...
- is not the global namespace and is an enclosing namespace of , ~~or~~
- is the parenthesized expression ,
- is a lambda capture of the closure type ,
- is the closure type of the lambda capture ,
- is the type specified by the *functional-type-conv-expression* ,
- is the function selected by overload resolution for a *function-call-expression* ,
- is the return type, parameter type, or function type of the function , or
- is reflection-related to an entity or alias and is reflection-related to .

For the operand , the type specified by the satisfies .

For an operand that is a parenthesized expression [expr.prim.paren], the type satisfies . For a parenthesized expression , whether or not itself nested inside a parenthesized expression, the expression shall be either a parenthesized expression, a *function-call-expression* or a *functional-type-conv-expression*; otherwise the program is ill-formed.

For an operand of the form *function-call-expression*, the type satisfies . If the *postfix-expression* of the *function-call-expression* is of class type, the function call shall not resolve to a surrogate call function ([over.call.object]). Otherwise, the *postfix-expression* shall name a function that is the unique result of overload resolution.

For an operand of the form *functional-type-conv-expression* [expr.type.conv], the type satisfies . [*Note:* The usual disambiguation between function-style cast and a *type-id* [dcl.ambig.res] applies. [*Example:* The default constructor of can be reflected on as , while reflects the type of a function returning . — *end example*] — *end note*]

> For an operand of the form *identifier* where *identifier* is a template *type-parameter*, the type satisfies both and .

Modify Table 12 as follows:

Category	***identifier* or *simple-template-id* kind**	**Concept**
type	*class-name* designating a union	
	class-name designating a closure type	
	class-name designating a non-union class	
	enum-name	
	type-name introduced by a *using-declaration*	both and
	any other *typedef-name*	both and
namespace	*namespace-alias*	both and
	any other *namespace-name*	both and
data member	the name of a data member	
value	the name of a variable or structured binding that is not a local entity	
	the name of an enumerator	both and
	the name of a function parameter	
	the name of a captured entity [expr.prim.lambda.capture]	

Modify the following paragraph as follows:

> If the *reflexpr-operand* designates an entity or alias at block scope (6.3.3) or function prototype scope (6.3.4) and the entity is neither captured nor a function parameter, the program is ill-formed. If the *reflexpr-operand* designates a class member, the type represented by the *reflexpr-specifier* also satisfies . If the *reflexpr-operand* designates an ~~variable or a data member~~ expression, it is an unevaluated operand (expr.context). If the *reflexpr-operand* designates both an alias and a class name, the type represented by the *reflexpr-specifier* reflects the alias and satisfies .

11 Declarators [dcl.decl]

No changes are made to Clause 11 of the Reflection TS.

12 Classes [class]

No changes are made to Clause 12 of the Reflection TS.

13 Derived classes [class.derived]

No changes are made to Clause 13 of the Reflection TS.

14 Member access control [class.access]

No changes are made to Clause 14 of the Reflection TS.

15 Special member functions [special]

No changes are made to Clause 15 of the Reflection TS.

16 Overloading [over]

No changes are made to Clause 16 of the Reflection TS.

17 Templates [temp]

No changes are made to Clause 17 of the Reflection TS.

18 Exception handling [except]

No changes are made to Clause 18 of the Reflection TS.

19 Preprocessing directives [cpp]

No changes are made to Clause 19 of the Reflection TS.

20 Library introduction [library]

No changes are made to Clause 20 of the Reflection TS.

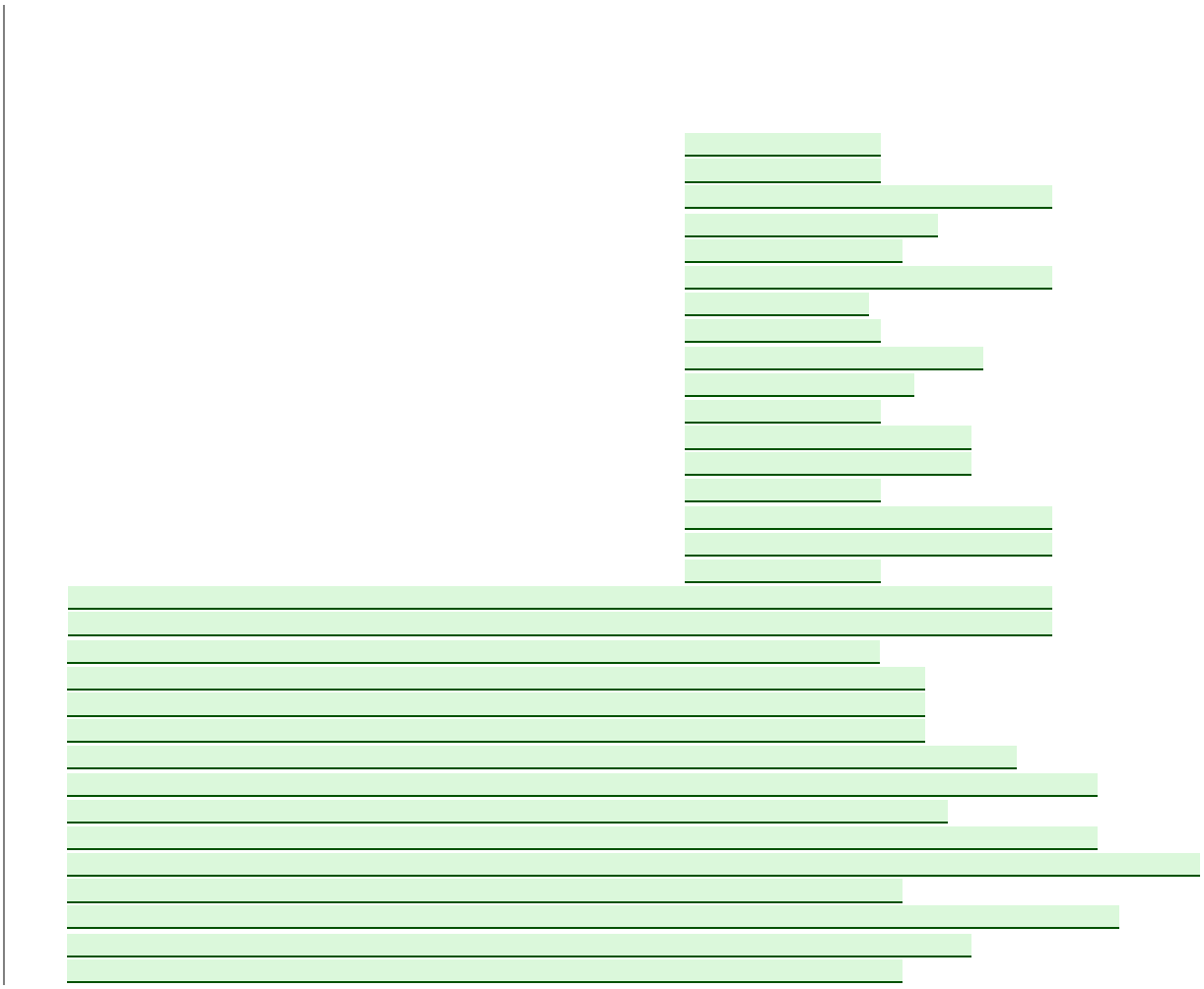
21 Language support library [language.support]

21.11 Static reflection [reflect]

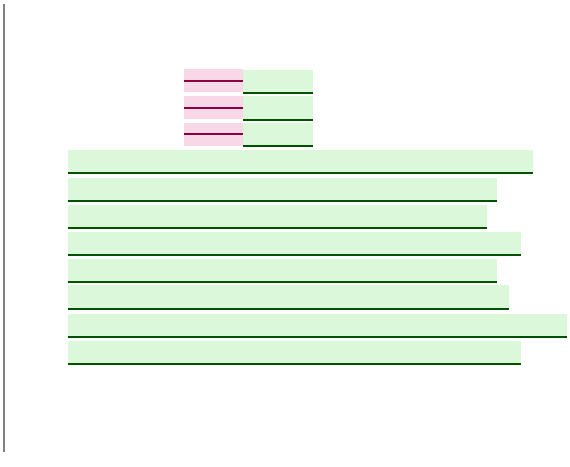
21.11.1 In general [reflect.general]

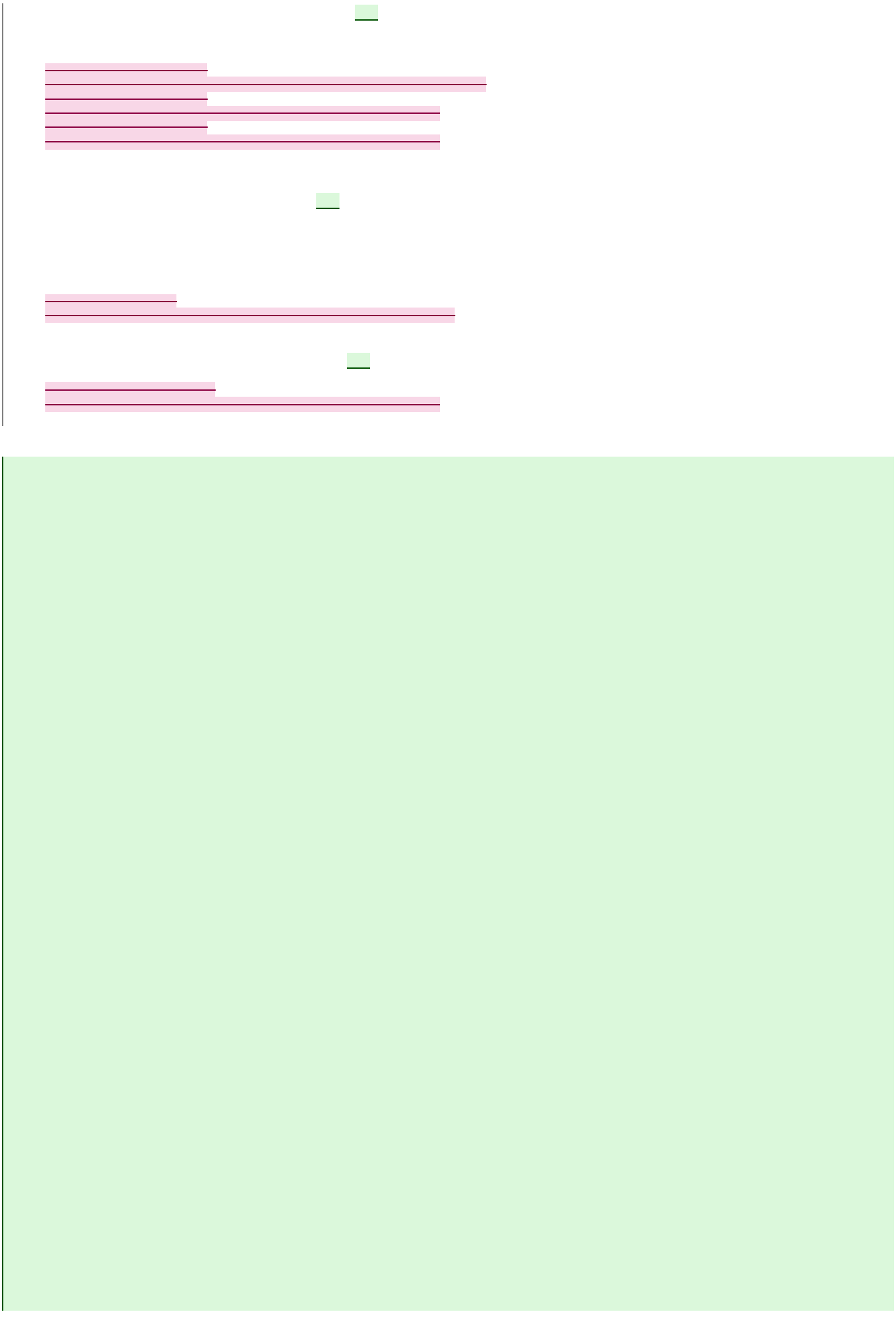
21.11.2 Header synopsis [reflect.synopsis]

Modify this section as follows:



...





21.11.3 Concepts for meta-object types **[reflect.concepts]**

Modify this section as follows:

21.11.3.1 Concept **[reflect.concepts.object]**

is ~~satisfied~~ if and only if is a meta-object type, as generated by the operator or any of the meta-object operations that in turn generate meta-object types.

21.11.3.2 Concept **[reflect.concepts.objseq]**

is ~~satisfied~~ if and only if is a sequence of s, generated by a meta-object operation.

21.11.3.3 Concept **[reflect.concepts.named]**

is ~~satisfied~~ if and only if ~~is an with~~has an associated (possibly empty) name.

21.11.3.4 Concept **[reflect.concepts.alias]**

is ~~satisfied~~ if and only if ~~is a that~~ reflects a declaration, an *alias-declaration*, a *namespace-alias*, a template *type-parameter*, a *decltype-specifier*, or a declaration introduced by a *using-declaration*. [*Note:* ~~Any such also satisfies~~

~~; its scope~~ The of an is the scope that the alias was injected into. *— end note*]

[*Example:*

The scope reflected by is , not . *— end example*]

Except for the type represented by the operator, properties resulting from type transformations (21.11.4) are not retained.

21.11.3.5 Concept **[reflect.concepts.recordmember]**

is ~~satisfied~~ if and only if reflects a *member-declaration*. ~~Any such also satisfies .~~

21.11.3.6 Concept **[reflect.concepts.enumerator]**

is ~~satisfied~~ if and only if reflects an enumerator. ~~Any such also satisfies and ; the~~ [*Note:* The of an is its type also for enumerations that are unscoped enumeration types. *— end note*]

21.11.3.7 Concept **[reflect.concepts.variable]**

is ~~satisfied~~ if and only if reflects a variable or non-static data member. ~~Any such also satisfies .~~

21.11.3.8 Concept **[reflect.concepts.scopemember]**

is ~~satisfied~~ if and only if satisfies , , or , or if reflects a namespace that is not the global namespace. ~~Any such also satisfies .~~ [*Note:* The scope of members of an unnamed union is the unnamed union; the scope of enumerators is their type. *— end note*]

21.11.3.9 Concept **[reflect.concepts.typed]**

is ~~satisfied~~ if and only if reflects an entity with a type. ~~Any such also satisfies .~~

21.11.3.10 Concept **[reflect.concepts.namespace]**

is ~~satisfied~~ if and only if reflects a namespace (including the global namespace). ~~Any such also satisfies .~~ [*Note:* Any such that does not reflect the global namespace also satisfies . *— end note*]

21.11.3.11 Concept **[reflect.concepts.globalscope]**

is ~~satisfied~~ if and only if reflects the global namespace. [*Note:* Any such ~~also satisfies ; it~~ does not satisfy . *— end note*]

21.11.3.12 Concept **[reflect.concepts.class]**

is ~~satisfied~~ if and only if reflects a non-union class type. ~~Any such . also satisfies .~~.

21.11.3.13 Concept **[reflect.concepts.enum]**

is ~~satisfied~~ if and only if reflects an enumeration type. ~~Any such . also satisfies . and .~~.

21.11.3.14 Concept **[reflect.concepts.record]**

is ~~satisfied~~ if and only if reflects a class type. ~~Any such . also satisfies . and .~~.

21.11.3.15 Concept **[reflect.concepts.scope]**

is ~~satisfied~~ if and only if reflects a namespace (including the global namespace), class, ~~or~~ enumeration, function or closure type. [*Note:* Any such that does not reflect the global namespace also satisfies . — *end note*]

21.11.3.16 Concept **[reflect.concepts.type]**

is ~~satisfied~~ if and only if reflects a type. ~~Any such . also satisfies . and .~~.

21.11.3.17 Concept **[reflect.concepts.const]**

is ~~satisfied~~ if and only if reflects a constant expression ([expr.const]). ~~Any such . also satisfies . and .~~.

21.11.3.18 Concept **[reflect.concepts.base]**

is ~~satisfied~~ if and only if reflects a direct base class, as returned by the template .

Add the following paragraphs at the end of [reflect.concepts]:

21.11.3.19 Concept **[reflect.concepts.fctparam]**

is if and only if reflects a function parameter. [*Note:* The of a is the to which this parameter appertains. — *end note*]

[*Note:* A does not satisfy , and thus does not offer an interface for getting the pointer to a parameter. — *end note*]

21.11.3.20 Concept **[reflect.concepts.callable]**

is if and only if reflects a function, including constructors and destructors.

21.11.3.21 Concept **[reflect.concepts.expr]**

is if and only if reflects an expression [expr].

21.11.3.22 Concept **[reflect.concepts.expr.paren]**

is if and only if reflects a parenthesized expression [expr.prim.paren].

21.11.3.23 Concept **[reflect.concepts.expr.fctcall]**

is if and only if reflects a *function-call-expression* [expr.call].

21.11.3.24 Concept **[reflect.concepts.expr.type.fctconv]**

is if and only if reflects a *functional-type-conv-expression* [expr.type.conv].

21.11.3.25 Concept **[reflect.concepts.fct]**

is if and only if reflects a function, excluding constructors and destructors.

21.11.3.26 Concept **[reflect.concepts.memfct]**

is if and only if reflects a member function, excluding constructors and destructors.

21.11.3.27 Concept **[reflect.concepts.specialfct]**

is if and only if reflects a special member function [special].

21.11.3.28 Concept **[reflect.concepts.ctor]**

is if and only if reflects a constructor. [*Note:* Some types that satisfy also satisfy . — *end note*]

21.11.3.29 Concept **[reflect.concepts.dtor]**

is if and only if reflects a destructor.

21.11.3.30 Concept **[reflect.concepts.oper]**

is if and only if reflects an operator function [over.oper] or a conversion function [class.conv.fct]. [*Note:* Some types that satisfy also satisfy or . — *end note*]

21.11.3.31 Concept **[reflect.concepts.convfct]**

is if and only if reflects a conversion function [class.conv.fct].

21.11.3.32 Concept [reflect.concepts.lambda]

is if and only if reflects a closure object (excluding generic lambdas).

21.11.3.33 Concept [reflect.concepts.lambdacapture]

is if and only if reflects a lambda capture as introduced by the capture list or by capture defaults. [*Note:* The of a is its immediately enclosing . — *end note*]

21.11.4 Meta-object Operations [reflect.ops]

Add two new final paragraphs to [reflect.ops]:

Some operations specify result types with a nested type called that satisfies one of the concepts in . These nested types will possibly satisfy other concepts, for instance more specific ones, or independent ones, as applicable for the entity reflected by the nested type. [*Example:*

While is specified to be a , will even satisfy . — *end example*]

If subsequent specializations of operations on the same reflected entity could give different constant expression results (for instance for because the parameter's function is redeclared with a different parameter name between the two points of instantiation), the program is ill-formed, no diagnostic required. [*Example:*

— *end example*]

21.11.4.1 Multi-concept operations [reflect.ops.over]

Remove the section 21.11.4.1.

21.11.4.4 Named operations [reflect.ops.named]

Modify the relevant part of [reflect.ops.over] as follows:

- - ...
  - for reflecting a class data member, its unqualified name~~.~~;
  - for reflecting a function, its unqualified name;
  - for reflecting a specialization of a template function, its *template-name*;
  - for reflecting a function parameter, its unqualified name;
  - for reflecting a constructor, the *injected-class-name* of its class;
  - for reflecting a destructor, the *injected-class-name* of its class, prefixed by the character '~';
  - for reflecting an operator function, the *operator* element of the relevant *operator-function-id*;
  - for reflecting an conversion function, the same characters as , with reflecting the type represented by the *conversion-type-id*.
- In all other cases (for instance for reflecting a lambda object), the string's value is the empty string for get_name<T> and implementation-defined for get_display_name<T>.

Insert a new final paragraph:

Remarks: Subsequent specializations of on the same reflected function parameter can render the program ill-formed, no diagnostic required (21.11.4).

21.11.4.7 Member operations **[reflect.ops.member]**

Modify the relevant part of [reflect.ops.over] as follows:

> With *ST* being the scope of the declaration of the entity ~~or~~, typedef or value reflected by , *S* is found as the innermost scope enclosing *ST* that is either a namespace scope (including global scope), class scope, ~~or~~ enumeration scope, function scope (for the function's parameters), or immediately enclosing closure type (for lambda captures). For members of an unnamed union, this innermost scope is the unnamed union. For enumerators of unscoped enumeration types, this innermost scope is their enumeration type.

21.11.4.8 Record operations **[reflect.ops.record]**

Modify the relevant part of [reflect.ops.record] as follows:

> All specializations of these templates shall meet the requirements ([meta.rqmts]). The nested type named is an alias to an specialized with types that reflect the following subset of non-template ~~data~~ members of the class reflected by :
>
> - for ( ), all data (function, including constructor and destructor) members.
> - for ( ), all public data (function, including constructor and destructor) members;
> - for ( ), all data (function, including constructor and destructor) members that are accessible from the context where the *reflexpr-specifier* appeared which (directly or indirectly) generated .
>
> [*Example:*
>
> — *end example*]
>
> The order of the elements in the is the order of the declaration of the ~~data~~ members in the class reflected by .
>
> *Remarks:* The program is ill-formed if reflects a closure type.

> All specializations of these templates shall meet the requirements ([meta.rqmts]). The nested type named is an alias to an specialized with types that reflect the following subset of function members of the class reflected by :
>
> - for , all constructors.
> - for , the destructor;
> - for , all conversion functions [class.conv.fct] and operator functions [over.oper].
>
> The order of the elements in the is the order of the declaration of the members in the class reflected by .
>
> *Remarks:* The program is ill-formed if reflects a closure type.

Modify the relevant part of [reflect.ops.record] as follows:

> All specializations of shall meet the requirements ([meta.rqmts]). If reflects a class that is marked with the *class-virt-specifier* , the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.10 Value operations **[reflect.ops.value]**

Modify the relevant part of [reflect.ops.value] as follows:

> All specializations of shall meet the requirements ([meta.rqmts]). If reflects a variable declared with the *decl-specifier* , the base characteristic of the respective template specialization is , otherwise it is .
>
> All specializations of shall meet the requirements ([meta.rqmts]). If reflects a variable with static storage duration, the base characteristic of the respective template specialization is , otherwise it is .
>
> All specializations of shall meet the requirements ([meta.rqmts]), with a static data member named of type and value , where
>
> - for variables with static storage duration: is , where is the type of the variable reflected by and is the address of that variable; otherwise,
> - is the pointer-to-member type of the member variable reflected by and a pointer to the member.
>
> [*Note:* A does not satisfy , and thus does not offer an interface for getting the pointer to a parameter. *— end note*]

21.11.4.11 Base operations **[reflect.ops.derived]**

Modify the relevant part of [reflect.ops.derived] as follows:

> All specializations of shall meet the requirements ([meta.rqmts]). The nested type named is an alias to , where is the base class reflected by .

21.11.4.12 Namespace operations **[reflect.ops.namespace]**

Modify the relevant part of [reflect.ops.derived] as follows:

> All specializations of shall meet the requirements ([meta.rqmts]). If reflects an inline namespace, the base characteristic of the template specialization is , otherwise it is .

Add the following paragraphs at the end of [reflect.ops]:

> **21.11.4.13 FunctionParameter operations** **[reflect.ops.fctparam]**
>
> All specializations of this template shall meet the requirements ([meta.rqmts]). If reflects a parameter with a default argument, the base characteristic of is , otherwise it is .
>
> *Remarks:* Subsequent specializations of on the same reflected function parameter can render the program ill-formed, no diagnostic required (21.11.4).
>
> **21.11.4.14 Callable operations** **[reflect.ops.callable]**

All specializations of this template shall meet the requirements ([meta.rqmts]). The nested type named is an alias to an specialized with types that reflect the parameters of the function reflected by . If that function's *parameter-declaration-clause* [dcl.fct] terminates with an ellipsis, the does not contain any additional elements reflecting that. The trait can be used to determine if the terminating ellipsis is in its parameter list.

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects an entity with an ellipsis terminating the *parameter-declaration-clause* [dcl.fct] (for), or an entity that is (where applicable implicitly or explicitly) declared as (for), (for), as an inline function [dcl.inline] (for), or as deleted (for), the base characteristic of the respective template specialization is , otherwise it is .

Remarks: Subsequent specializations of on the same reflected function can render the program ill-formed, no diagnostic required (21.11.4).

21.11.4.15 ParenthesizedExpression operations **[reflect.ops.expr.paren]**

All specializations of shall meet the requirements ([meta.rqmts]). The nested type named is the type reflecting the expression of the parenthesized expression reflected by .

21.11.4.16 FunctionCallExpression operations **[reflect.ops.expr.fctcall]**

All specializations of shall meet the requirements ([meta.rqmts]). The nested type named is the type reflecting the function invoked by the *function-call-expression* which is reflected by .

21.11.4.17 FunctionalTypeConversion operations **[reflect.ops.expr.fcttypeconv]**

All specializations of shall meet the requirements ([meta.rqmts]). For a type conversion reflected by , the nested type named is the reflecting the constructor of the type specified by the type conversion, as selected by overload resolution. The program is ill-formed if no such constructor exists. [*Note:* For instance fundamental types (6.7) do not have constructors. — *end note*]

21.11.4.18 Function operations **[reflect.ops.fct]**

All specializations of shall meet the requirements ([meta.rqmts]), with a static data member named of type and value , where

- for non-static member functions, is the pointer-to-member-function type of the member function reflected by and a pointer to the member function; otherwise,
- is , where is the type of the function reflected by and is the address of that function.

21.11.4.19 MemberFunction operations **[reflect.ops.memfct]**

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects a member function that is (for), (for), (for), declared with a *ref-qualifier* (for) or (for), implicitly or expicitly (for), pure virtual (for), or marked with (for) or (for), the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.20 SpecialMemberFunction operations **[reflect.ops.specialfct]**

All specializations of these templates shall meet the requirements ([meta.rqmts]). If their template parameter reflects a special member function that is implicitly declared (for) or that is defaulted in its first declaration (for), the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.21 Constructor operations **[reflect.ops.ctor]**

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an explicit constructor, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.22 Destructor operations **[reflect.ops.dtor]**

All specializations of these templates shall meet the requirements ([meta.rqmts]). If the template parameter reflects a virtual (for) or pure virtual (for) destructor, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.23 ConversionOperator operations **[reflect.ops.convfct]**

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an explicit conversion function, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.24 Lambda operations **[reflect.ops.lambda]**

All specializations of this template shall meet the requirements ([meta.rqmts]). The nested type named is an alias to an specialized with types that reflect the captures of the closure object reflected by .

The elements are in order of appearance in the *lambda-capture*; captures captured because of a *capture-default* have no defined order among the default captures.

All specializations of these templates shall meet the requirements ([meta.rqmts]). If the template parameter reflects a closure object with a *capture-default* that is (for) or (for), the base characteristic of the respective template specialization is , otherwise it is .

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects a closure object with a function call operator, the base characteristic of the respective template specialization is , otherwise it is .

21.11.4.25 LambdaCapture operations **[reflect.ops.lambdacapture]**

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an explicitly captured entity, the base characteristic of the respective template specialization is , otherwise it is .

All specializations of this template shall meet the requirements ([meta.rqmts]). If the template parameter reflects an *init-capture*, the base characteristic of the respective template specialization is , otherwise it is .

Revision history

Revision 3 (P0670R3)

First revision with formal wording.

Changes first appearing in P0670R4

- Identified "constructor calls" as .
- Introduced as disambiguation mechanism (type-id vs cast expression).
- Introduced *functional-type-conv-expression*.
- Objects with function call operator are not s anymore.
- Better definition of concepts, enabling subsummation and clarifying that all concepts have only syntactic (i.e. checkable) requirements.
- Specify the odr-rule behavior.
- Specify that lambda captures used as *reflexpr-operand* are captured.
- Repeated specializations of operations yielding different constant expression results are now ill-formed ndr.
- Specify that creates an odr-use.
- Replaced ellipsis by operation.
- Rename to .

References

1. *Static reflection. Rationale, design and evolution.* p0385

2. *Static reflection in a nutshell.* p0578